

CIP Country Score Sync Script

系統設計流程與維護說明文件

一、整體設計目的

本資料庫指令碼 (Script) 的核心目的，在於以基準範本 **CIPT1200059 (Individual Customers)** 的「國家風險分數」為單一事實來源 (Single Source of Truth)，自動將其分數同步至多個結構相似的 CIP Template (如 `lkAnyCustomerComboCtry`、`lkMainBusinessCtry`、`lkAnyLegalAgentComboCtry` 等)。透過此自動化同步機制，確保系統內各層級之國家風險評分標準保持嚴謹的一致性。

同時本指令碼亦負責處理以下核心邏輯：

- **動態情境同步**：自動辨識並新增/更新各 Template 下對應的國家 Scenario 與分數。
- **落後資料清理**：針對來源範本已移除或不再需要的舊情境，執行軟刪除 (Soft Delete)。
- **暫存緩衝同步**：將更新同步至 Pending / MakerChecker 輔助表格，以利後續簽核流程與前端介面呈現。

二、完整執行流程與核心設計

Step 1: 設定參數與取得來源 Group

宣告核心變數並定義「以哪個 Template 與 Group 作為基準來源」。

```
DECLARE @CIPQueryId VARCHAR(50) = 'CIPT1200059'  
DECLARE @SampleCIPQueryGroupName NVARCHAR(50) = 'Individual Customers'
```

- 透過基準 Template 識別碼撈取其所屬的 @SourceGroupID。
- 判斷該 Group 類別為個人 (P) 或法人 (N)，藉此動態決定後續更新 INDV_SCORE 還是 BUSS_SCORE。

設計考量：堅持「單一來源基準」設計，杜絕各相似範本因人工調整而產生評分歧異的風險。

Step 2: 建立來源分數暫存表 #CountryScore_Script

從國家維度表與情境庫中，將基準範本目前於主表生效的最新國家分數完整提取。

- **核心機制**：使用 MAX() 配合 CASE WHEN 根據客群分類 (個人/法人) 動態抓取分數，並落入暫存表。
- **篩選控制**：僅抓取 DELETEFLAG <> 'Y' 的有效資料，避免過期或已刪除的分數遭到誤用。

Step 3: 建立理想情境暫存表 #CIP_IdealSet

此步驟為整體架構中最關鍵的「規格定義層」，指令碼在此進行兩次 INSERT 動作：

1. **第一次寫入**：針對 9 個主要的 ComboCtry 核心 Template 進行對應與開展。
2. **第二次寫入**：針對 `lkAnyOtherRelPartyComboCtry` 處理關係人 (Related Party) 的特殊情境對應。

設計目的：透過多表交叉聯結，為「每個目標 Template × 每個 ISO 國家」預先建立一筆理想中應存在的 Scenario 藍圖，並同時關聯出對應的 HRSTemplateId，作為後續比對與增刪的絕對標準。

Step 4: 安全備份機制

考量到後續將進行大量資料之軟刪除、更新與新型態寫入，指令碼具備自動動態備份機制：

- 利用當前日期 (YYYYMMDD) 動態命名並建立備份表：
CIP_SCENARIO_LIBRARY_BackUp_CountryDuplicate_2026xxxx。
- 同步針對 HRS_QuerySyntaxes 進行完整鏡像備份，確保營運環境之資料具備遭遇異常時可立即回滾 (Rollback) 之能力。

Step 5: 執行過期資料軟刪除 (Soft Delete)

比對主表與 Step 3 建立的理想集合 #CIP_IdealSet。若主表中特定目標 Template 下的國家 Scenario 「已不存在於理想集合中」，則判定該國家資料已失效。

處理手段：執行 UPDATE A SET A.DELETEFLAG = 'Y' 進行軟刪除，不直接調用 DELETE，保留資料稽核軌跡 (Audit Trail)。

Step 6: 遊標逐筆比對與異動處理 (Cursor Engine)

宣告 ScenarioCursor 遊標，將理想集合 #CIP_IdealSet 逐行登錄並與現存主表資料比對：

- **條件分支 - 資料存在：**若主表中該 Template、Group、國家代碼皆比對成功且為有效狀態，則執行 UPDATE，更新評級分數，並寫入系統管理員更新標記。
- **條件分支 - 資料不存在：**若主表查無此情境，則判定為新增國家或新範本開展。此時調用系統儲存程序 EXEC SP_BSAGETID 'CIP', 'U', ... 生成唯一序號 @NewQryID，並執行 INSERT 寫入全新 Scenario。

※ CIP_SCENARIO_LIBRARY 與 HRS_QuerySyntaxes 兩張核心表皆在此循環中同步處理完成。

Step 7: 緩衝表刷新與同步

為使 Maker-Checker (經辦-覆核) 與前端 UI 介面能夠即時呈現最新且一致的資料狀態，指令碼在主表更新完畢後執行末端同步：

```
TRUNCATE TABLE CIP_SCENARIO_LIBRARY_Pending
INSERT INTO CIP_SCENARIO_LIBRARY_Pending SELECT * FROM CIP_SCENARIO_LIBRARY
```

此步驟會徹底清空 Pending 表並重新由主表導入，以確保兩端資料完全一致。

三、整體設計架構圖

- 來源 Template (CIPT1200059) — 核心基準事實來源
- 國家分數提取 (#CountryScore_Script) — 區分個人/法人抓取有效分數
- 建立規格藍圖 (#CIP_IdealSet) — 交叉聯結生成 [Template × 國家 × 分數] 理想集合
- 執行安全備份 — 動態日期命名，鏡像備份 CIP 與 HRS 核心表
- 邊界清理 (Soft Delete) — 比對 IdealSet，將不在預期內的舊資料標記為 DELETEFLAG = 'Y'
- 核心遊標引擎 (Cursor Engine) — 逐筆判斷：
 - 存在 → 執行 UPDATE 更新分數與審查軌跡
 - 不存在 → 調用 SP_BSAGETID 生成新 ID 並執行 INSERT
- Pending 表格同步 — TRUNCATE 後全量導入，供 Maker-Checker 機制與 UI 展現

四、當前設計風險與潛在問題分析

經對現行指令碼邏輯進行代碼評審（Code Review），發現以下於特定營運情境下可能引發異常之設計風險：

問題點	嚴重程度	技術說明與影響
GroupId 聯結解析不精準	高風險 (High)	在建立 #CIP_IdealSet 時，僅透過 Category 去 JOIN CIP_SyntaxGroups，未限縮必須與來源組名一致 (GroupName = @SampleCIPQueryGroupName)。這將導致產生預期之外的 GroupId 組合，引發後續 Cursor 比對失真，持續走入 ELSE 分支，進而產生大量重複資料 (Duplicate)。
重複執行累積髒資料	高風險 (High)	由於 Step 5 的 Soft Delete 排除條件與 Step 6 的 EXISTS 檢查若因上述 GroupId 錯置而失準，每次重複執行該 Script 時，系統將無法正確識別已存在記錄，導致 SP_BSAGETID 不斷被觸發，造成資料庫容量無端膨脹與查詢效能下降。
未納入 Pending 分數考量	中風險 (Medium)	本指令碼之分數完全依賴「主表」的歷史資料。若經辦人員於 UI 介面修改了分數且該資料尚處於 Pending (未覆核) 階段，該調整將無法被此指令碼捕捉，可能導致覆核中的新分數遭到主表舊分數覆蓋。
缺乏動態日誌與運作回報	中風險 (Medium)	指令碼執行完畢後僅顯示常規完成訊息，缺乏結構化日誌 (Log)。維運人員無法得知本次執行具體影響的筆數 (更新幾筆、新增幾筆、軟刪除幾筆)，不利於生產環境的變更稽核。
缺乏事務 (Transaction) 保護	中風險 (Medium)	指令碼中未顯式宣告 BEGIN TRANSACTION 與 COMMIT / ROLLBACK。若在 Cursor 執行中途 (例如某一筆寫入因違反限制而報錯)，將會導致資料庫處於半同步的「斷裂狀態」，增加人工修復成本。

五、建議優化與演進方向

為提升此自動化腳本的健壯性 (Robustness) 與維運安全性，建議於後續變更中導入以下優化方針：

- 精準化 Group 關鍵字篩選：**於 #CIP_IdealSet 的 JOIN 條件中，強制限縮 AND B.GroupName = @SampleCIPQueryGroupName，自源頭切斷非預期 GroupId 的引入。
- 增設自動化日誌統計表：**建立技術統計變數 (如 @UpdateCount, @InsertCount)，於 Cursor 各分支進行累加，並於指令碼末端將統計結果 INSERT 至維運日誌表，落實營運軌跡追蹤。
- 事務完整性包覆：**將 Step 5 至 Step 7 的核心異動區塊納入 BEGIN TRY ... BEGIN TRANSACTION ... COMMIT 區塊中，並於 CATCH 區塊內強制執行 ROLLBACK，確保全有或全無 (Atomicity) 的事務原則。
- 支援 Dry-Run (模擬執行) 模式：**增設 @IsDryRun BIT = 1 參數。當開關開啟時，僅 SELECT 出將被變更的資料規格與筆數，不真實寫入資料庫，供工程人員於上線前進行最終驗證。